Project 2 : Planetesimal Accretion Modeled Using Python

Course: PHY 607 Date: 10/22/2026

Authors: Kaitlyn Khalawan and Rachel Hevey

Introduction:

Planetesimal accretion is a proposed method of planet formation in which planets grow slowly through the gradual accumulation of matter due to gravity and collisions. This happens within a protoplanetary disk, a flat disk of dense gas and dust that orbits a newly formed star. Turbulence within the disk promotes collisions among the particles, while electrostatic forces promote the adhesion of colliding particles.

We approach this as an N-body problem. An N-body problem is a generalization of the classic three-body problem, which asks for the future positions and velocities of three point masses interacting through their mutual gravitational pull. A three-body problem consists of three position vectors (r12, r23, and r31), three masses (m1, m2, and m3), and the gravitational constant. These variables are used to find the gravitational force of one point mass on another. From there, one can use time intervals to find the change in momentum, update the momentum using the momentum principle, and finally use the definition of average velocity to update the position. Adding a fourth point mass to the system adds three more position vectors, and the number of interactions grows rapidly as more point masses are added (quadratically, for N bodies). Instead of calculating the interactions of each individual mass by hand, we use Python to loop through this process for us.

Using the N-body simulation as a base, we built a Python script that models planetesimal accretion. Each particle (dust, pebble, or protoplanet) is treated as a point mass, with a larger point mass at the center representing the star. From there, we add interactions between the protoplanets and collisions that end in either merging or bouncing.

The goal of this model was to determine the cumulative mass distribution before and after evolution.

The Code - High level algorithms:

Within the collisions class there are two high-level algorithms, the advance method and the handle collisions method. The function making an instance of the collision class , main, is also a high-level algorithm.

Advance Method:
This method is the event-driven loop at the center of the code. It integrates all living bodies with solve_ivp stopping when a collision event is triggered. After a trigger event is triggered it gets passed along to get resolved then repacked in an array to account for a possible change in the number of bodies. Once resolved this repeated for the remainder of the time left.

Handel Collisions Method:
This is the resolution loop that finds instances of overlapping protoplanets, measures the ratio between the relative velocity and the escape velocity, makes a random decision on the collision outcome between merge and bounce, logs it, and then repeats until no overlaps remain.

Main Function:
This is the top-level flow, beginning with the start of the run in a specific method (animation, headless, integration check, or benchmark), propagating the initial conditions, and calling the advance method repeatedly until the end of the designated time, then reports the results in both graphical format and printed in the terminal.

Results:

Our N variable sets the number of bodies scattered over the fixed ring (0.8 - 1.5 AU) surrounding the star, in other terms N sets the surface density and total mass of the system. One of the results that scales with N is the collision rate with higher surface density leading to more collisions. We ran the simulation with N values of 50, 100, and 200 and recorded the number of collisions, the percentages of merges and bounces, and the total mass, as detailed in Table 1. The data show that both the number of collisions and the total mass increase with N, while the percentage of collisions ending in a merge or a

bounce shows no clear trend. This data set would be greatly improved by an expansion in the selected N values and more runs for each N value. While increasing the number of runs is an easy increase, the increase in N value comes with some additional challenges such as increased cost to run the program and an increased risk of bodies' initial positions being overlapping which can cause issues with the code correctly identifying collision events.

Table 1: Comparison of the number of collisions, merge and bounce percentages, and total mass for varying initial surface densities. Values were averaged over three runs with randomized initial conditions and different seeds.

N	Number of Collisions	Merge Percent	Bounce Percent	Total Mass (Msun)
50	115	1.74%	98.26%	1.438505e-06
100	333	2.10%	97.90%	2.799143e-06
200	1279	1.41%	98.59%	5.100692e-06

Validation:

Merge Only and Bounce Only:
When the simulation is set to have every collision result in a merger of the two bodies the mass and linear momentum of the bodies should be conserved. To check this we ran the simulation with the merge probability set to one and the bounce probability set to zero. When averaged across three collisions the sum of the mass of the two bodies prior to the collision was equal to the mass of the body created by the merger. This validates that the code is correctly managing the masses of combined bodies which is further supported by the total mass of the system staying constant throughout the simulation.

When the simulation is set to have every collision result in the two bodies bouncing off of each other the momentum of the bodies should be conserved. To check this we ran the simulation with the probability of a bounce set to one and the probability of a merge set to zero. When the total momentum for each participating body was calculated using the 3D Pythagorean theorem, $p_{total} = \sqrt{(p1)^2 + (p2)^2 + (p3)^2}$, and averaged over three runs the change in momentum from before the collision to after the collision was zero. This

validates that the code is correctly simulating an inelastic collision, like the one we would expect from two colliding bodies that bounce off of one another.

Since both collision types are inelastic collisions the kinetic energy should decrease. The kinetic energy decrease should be more prominent for a merge than for a bounce due to the shared velocity of the post merge bodies.To validate the decrease in kinetic energy after each collision, merges and bounces were turned back to their original probabilities and the kinetic energy lost per collision was plotted against the ratio between relative velocity and escape velocity at each collision, this ratio is the same one used in the function to randomly assign the outcome of the collision. Figure 1 shows that mergers dissipate the full relative kinetic energy, while bounces dissipate a fraction $(1-e^2)$ of the normal component, as expected from the collision rules.

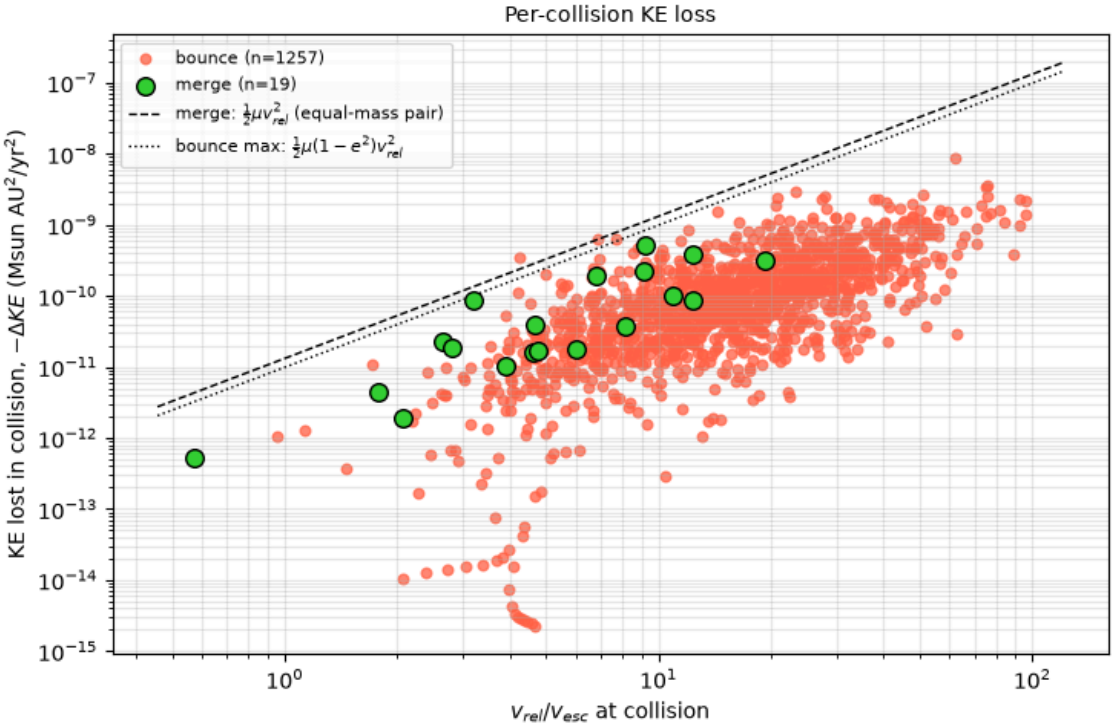
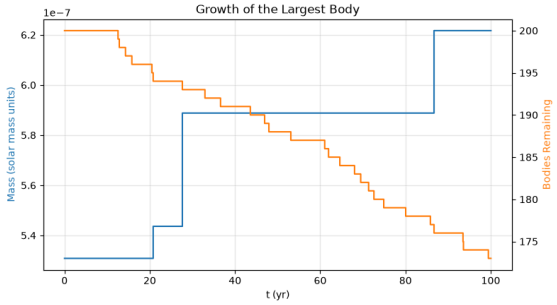


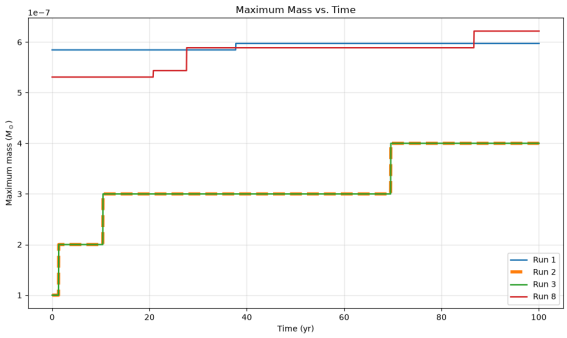
Figure 1. Kinetic energy lost per collision versus v_rel/v_esc for N = 200 planetesimals over 100 yr with a seed of 42. Mergers (green, n = 19) and bounces (red, n = 1257) all lose energy. Dashed and dotted lines show the loss in an equal-mass merger and the maximum loss in an equal-mass head-on bounce, respectively.

Total Mass Conservation:

Since we are running a contained system and the only collision outcomes are the bodies merging or the bodies bouncing with no exchange of mass, the total mass of the system should remain constant while the mass of the current largest body should only ever increase as we progress through time. To check this verification the total mass of the system was reported at the start and end of the simulation, over multiple runs with different randomization seeds the value stayed constant. To check the growth behavior the largest body (by mass) was tracked throughout the run and plotted against time in figure 2, after each merger the new body was compared to the previous largest mass and the largest mass was updated accordingly. The figure also shows the inverse relationship between the mass of the largest body and the remaining number of bodies in the system which is expected as a body is lost from the system every time there is a merge. The validating part of this information is the gradual increase in the mass of the largest body with no areas of decrease.



a)



b)

Figure 2: a) The mass of the current largest body and the number of bodies in the system compared to time for N = 200 planetesimals over 100 yr with a seed of 4. Depicting a direct relationship between the mass of the largest body and time as well as an inverse relationship between the number of bodies and time. b) The average growth of the largest body over four different seeds compared to time, each seed ran for N = 200 planetesimals over 100 yr.

Validation of Kepler's 3rd Law:

Kepler's Third Law states that $T^2 \propto a^3$, which denotes that the square of the orbital period is proportional to the cube of the semi-major axis of an ellipse. In units of AU, years, and solar masses, this becomes $T^2 = a^3$.

The validation arose from a single body following a circular orbit around 1 central solar mass with $G = 4\pi^2$. The single body was set up with a small mass of $1 * 10^{-12}$ and a radius of $1 * 10^{-9}$, making it effectively massless. In addition, its initial position on the x-axis as it then orbits in the x-z plane. Four orbits were run, with a fixed timestep of 0.002 years for 4 expected periods each. The period was then measured by recording the accumulating polar angle and interpolating the time at which each full revolution was completed.

The results were expected to follow the law that $T = a^{1.5}$ and the relationship would yield a slope of 1. This was then reflected in the run generating *Figure* 3. This shows that the simulation reproduces Kepler's Third Law, where the largest orbit of 4 AU is 8 times bigger than the smallest of 0.5 AU, and is not due to potential, extraneous variables. The small mass of the single body also allows it to avoid interfering with the central body it orbits.

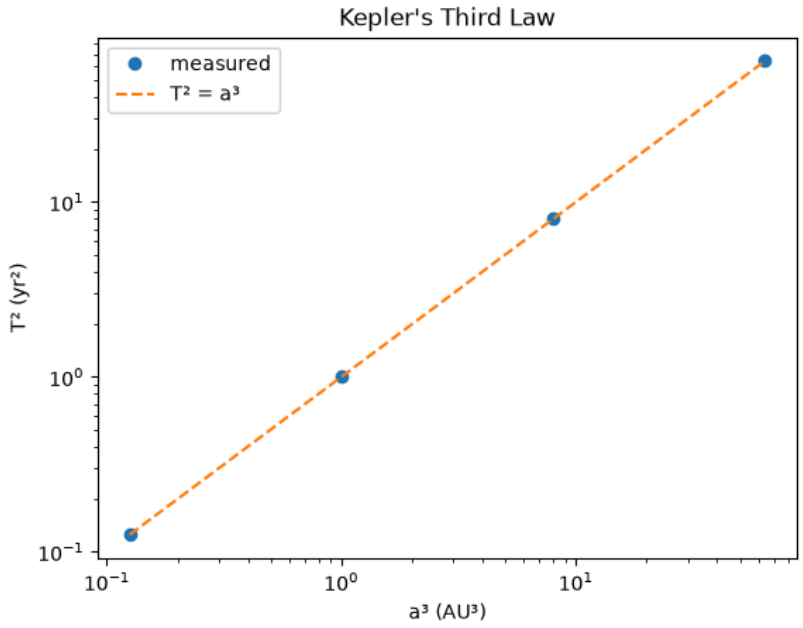


Figure 3. Validation of the simulation's orbital dynamics against Kepler's third law. The square of the orbital period (T^2, yr^2) is plotted against the cube of the semi-major axis (a^3, AU^3) on logarithmic axes for isolated test bodies orbiting a one-solar-mass star. Measured values (blue points) agree with the theoretical relation $T^2 = a^3$ (dashed orange line) across more than two orders of magnitude.

Errors:

The initial conditions of our system are drawn from pseudo-random distributions so they contain statistical (sampling) error. Any single run is one realization of the ring, not a depiction of the actual ring itself. We used three kinds of random draws inverse CDF for masses and rejection sampling for positions.

Masses - Inverse CDF:
Each protoplanet mass is drawn from a power law, $dN/dm \propto m^{-2.5}$, between 10^{-8} and 10^{-6} solar mass units. We draw a uniform number U and map it through the inverse of the cumulative distribution. With N = 100 bodies the sample only approximates the target distribution. Due to the fact that the exponent is steep, most bodies are small and the massive tail is thinly sampled. Across 2,000 seeds, the total ring mass averaged $2.7 * 10^{-6}$ solar mass units with a scatter of about 16% from seed to seed. The most massive initial body ranged from $1.0 * 10^{-7}$ to $7.6 * 10^{-7}$ solar mass units. Since the heaviest bodies dominate growth, this tail decides how quickly a runaway body can form.

Positions - Rejection Sampling:
Positions come from a surface density $\Sigma \propto r^{-1.5}$ over an annulus from 0.8 to 1.5 AU. We draw (x, z) uniformly in a square, only keep the point if it lies in the annulus, and accept it with probability $r^{-1.5} / r_{in}^{-1.5}$, This is the "unnormalized" part: rejection sampling only needs the density up to a constant, so we never normalize Σ. The accepted points follow the correct distribution in the limit, so the error is not a bias. It is finite-sample noise. With only 100 points the ring is lumpy, with random clusters and gaps, and the mean orbital radius shifts between seeds (about ±0.02 AU). Close pairs in a cluster collide early, which affects the early collision counts. The method is also inefficient, since only about 33% of proposals are accepted. That costs run time but does not add error. On top of this because initial positions were drawn independently, about 65% of rings started with at least one pair of

overlapping bodies. The solver's contact event only detects gaps that go from positive to negative, so these pairs were never registered as collisions. This slightly undercounts early collisions (about 1% or less of the total) and means those pairs briefly interpenetrate under softened gravity.

Since these draws are pseudo-random, they come from a deterministic generator and are reproducible for a fixed seed, but the physical outcome from one seed is still only one sample. Results such as 330 collisions and 10 merges should be treated as one sample from a distribution. The way to reduce this error is to run many seeds and report the mean and standard deviation. The uncertainty on the mean shrinks as $1 / \sqrt{(number\ of\ runs}$ and the ring itself is better resolved with larger N values. Sadly in this case a smaller step size won't reduce this error, In this code step size only controls integration error and DOP853 is adaptive, so it is tightened through rtol (1e-9) and atol (1e-12), not a fixed step size.

Code Benchmarks:

After using the timeit python library to run a benchmarking function, we were able to see a bottleneck in the code. Within the collision class there is a bottleneck at the rhs method with an alarming 592.2 μs cost per call at an N value of 100. This was over double the next highest cost per call, which was the find overlap method coming in at 240.4 μs. This is expected, since the cost likely comes from how the pairwise gravity is computed. Every pair of particles is evaluated in three dimensions, so at N = 100 the displacement array has shape (100, 100, 3), or 30,000 numbers, and the expression creates several arrays of that size. This is why the cost grows about fourfold when N is doubled, consistent with $O(N^2)$ scaling. The method also holds several full-size temporaries: three arrays are created and then reduced, and each one allocates memory, is filled, and is read back, which makes memory traffic a limiting factor. A faster version could compute a single (N, N) inverse-cube array and then finish with matrix products instead of an (N, N, 3) sum, which would reduce the number of temporaries created.